

Model-View-Controller

A very popular architectural pattern

Contents

1. MVC Architectural Pattern Overview
2. How MVC works
3. Basic MVC (Beginner Level)
4. MVC + DAO/DTO (Intermediate Level)
5. MVC + Service Layer (Professional Level)

1. MVC Architectural Pattern Overview

What are Architectural Pattern used for?

- Define high-level solutions for structuring software systems
- Provide reusable solutions to common design problems
 - Separate concerns has responsible significant specific
 - Manage data flow more easy , easy handle
 - Handle scalability
 - Organize complex systems

view: is UI (i/o)

Controller: is take request from view, process in model, then response to view

1. MVC Architectural Patterns Overview (cont.)

Model: is hadle data Manipulation and Interaction, businssess logic.

MVC Architectural Pattern:

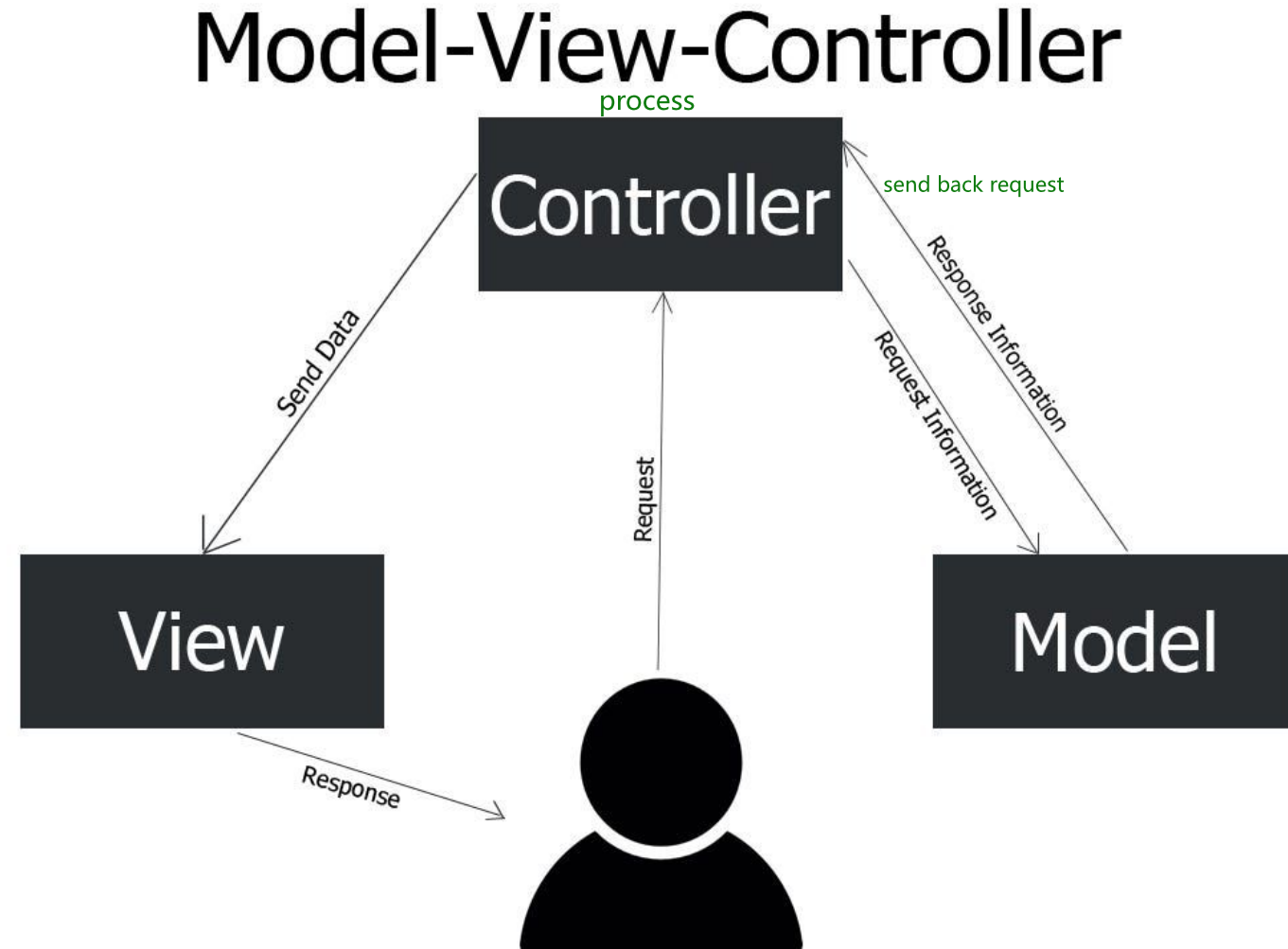
- MVC stands for Model-View-Controller is response for processing
- An architectural design pattern used for developing web applications and software systems
- Divides an application into three components:
 - Model - handles data and business logic
 - View - handles UI and presentation
 - Controller - handles user input and application flow

2. How MVC works

Responsibility of the main MVC components:

- **Model** contains the application's data and main functionality.
- **View** displays the data to the user in the desired format (HTML, IMAGE, XML, JSON) and interacts with the user.
- **Controller** handles requests, user input and any action perform in the front-end, interacting between the model and the view.

2. How MVC works



2. How MVC works

There are a few disadvantages of MVC architecture:

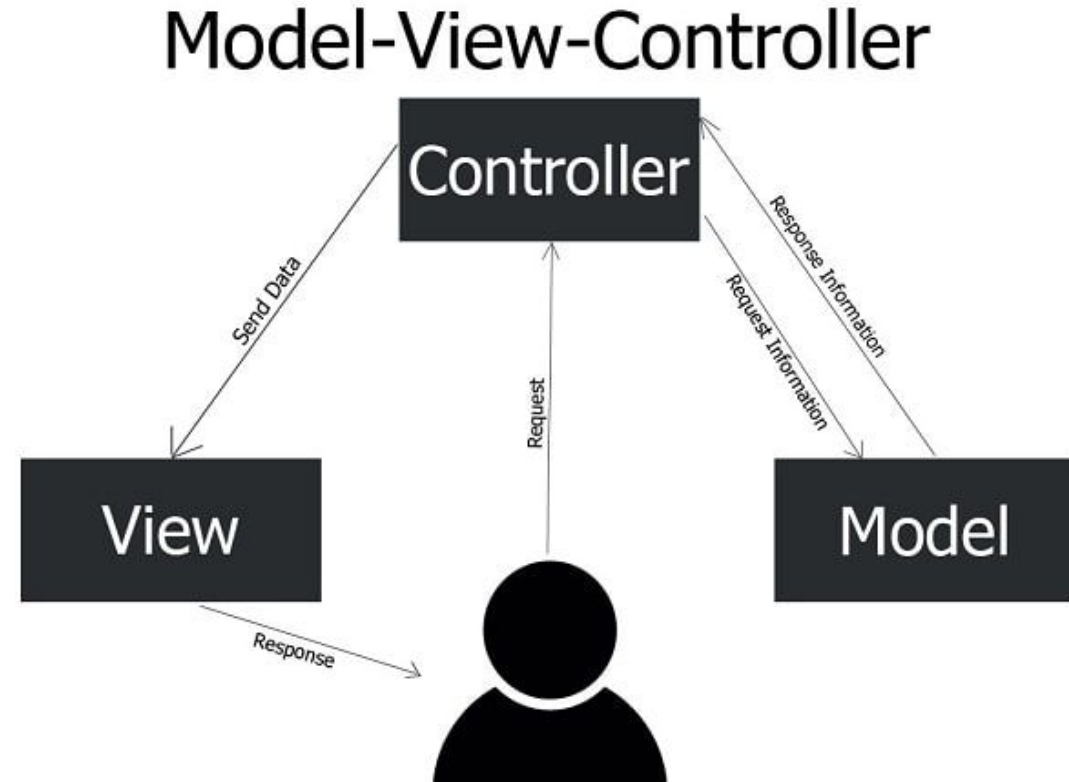
- Increase complexity and can be difficult to understand, especially for beginners.
- Changes may require updates across multiple layers (Model, View, Controller), which can increase maintenance effort.
- Controller must be carefully designed to avoid becoming overly complex requiring developers to follow strict architectural discipline.
- Requires developers to have a good understanding of architectural patterns and framework conventions.

However, these disadvantages are generally outweighed by the benefits of MVC, such as improved maintainability, scalability, and separation of concerns.

3. Basic MVC (Beginner Level)

Components:

- **Model** – holds data
- **View** – UI / presentation
- **Controller** – handles input and coordinates Model $\leftrightarrow$ View



4. MVC + DAO/DTO (Intermediate Level)

- **DAO (Data Access Object)** encapsulates the logic for retrieving, saving and updating data in your data storage (a database, a file-system, whatever).
- **DTO (Data Transfer Object)** is used to transfer data between layers in your application.
- ***MVC is a higher-level architectural pattern. Within MVC, DAO and DTO belong to the Model layer.***
- This pattern helps separate low-level data access operations from high-level business logic, improving maintainability and structure.

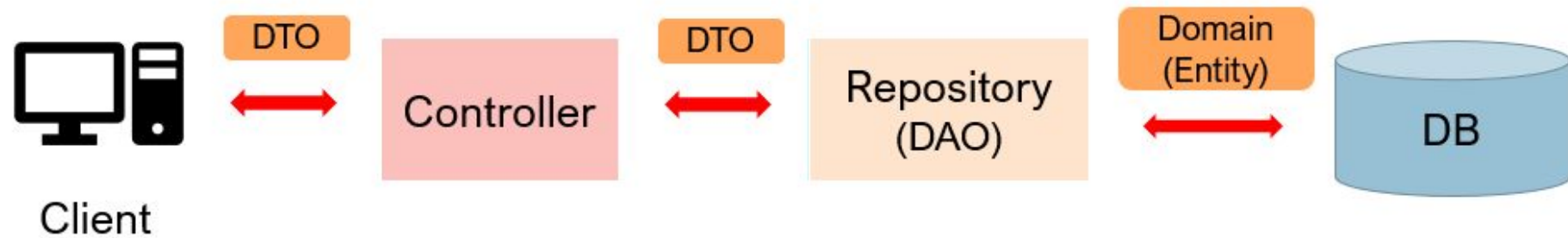
4. MVC + DAO/DTO (Intermediate Level)

Best practice for DAO/DTO Pattern

- **Data Access Object Interface** — Defines the standard operations to be performed on a model object(s).
- **Data Access Object Concrete Class** — implements the above interface.
- **Model Object** — A simple **POJO (Plain Old Java Object)** containing get/set methods to store data retrieved using the DAO class.
- **Data Transfer Object** — A data class contains final value object for providing to controller.

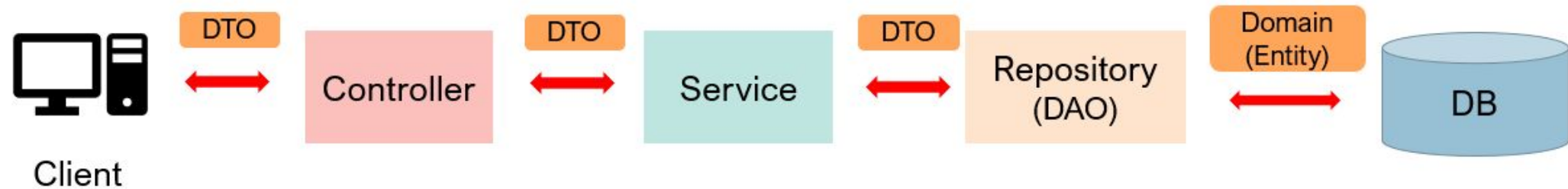
4. MVC + DAO/DTO (Intermediate Level)

How DAO and DTO pattern works:



5. MVC + Service Layer (Professional Level)

Including Service layer to DAO and DTO pattern:



5. MVC + Service Layer (Professional Level)

Final Project structure:

```
src/  
├── model/  
│   └── User.java  
├── dto/  
│   └── UserDto.java    (record)  
├── mapper/  
│   └── UserMapper.java  
├── dao/  
│   ├── UserDao.java  
│   └── UserDaoImpl.java  
├── service/  
│   ├── UserService.java  
│   └── UserServiceImpl.java  
├── controller/  
│   └── UserController.java  
├── view/  
│   └── UserView.java  
├── db/  
│   └── UserDb.java  
└── Main.java
```

add package Exception pkg

A large, faint watermark of the ISTAD logo is centered in the background. It is a circular emblem with a blue outer ring containing the text 'INSTITUTE OF SCIENCE AND TECHNOLOGY ADVANCED DEVELOPMENT' and Khmer text. Inside the ring is a blue circle with a white atomic symbol and binary code '01010100 01000001 01000000'.

Thank you

Begin with the theory, Start with the practical